Title: The Open Source Audit — Python
Your Name: AJAAN ALI
Registration Number: 24BAC10050
Course: Open Source Software
University: VIT Bhopal
Date: 31-03-2026

---

1. Introduction

Open source software is everywhere. If you've ever used a web browser, sent an email, or scrolled through social media, you've likely used open source technology. But what does "open source" really mean? At its core, open source means that the source code of a piece of software is made available to the public. Anyone can look at it, modify it, and distribute it. This model has given us some of the most important tools of the digital age: the Linux operating system, the Apache web server, the Firefox browser, and of course, the Python programming language.

Why is open source so important? First, it promotes collaboration. Thousands of developers from around the world can work together on a single project, sharing ideas and improvements. Second, it ensures transparency. When code is open, it can be inspected for security vulnerabilities or privacy issues. Third, it reduces costs. Businesses and individuals can use high-quality software without paying licensing fees. Finally, it gives users freedom—the freedom to run, study, change, and share the software as they see fit.

Python is one of the most successful open source projects ever. Created by Guido van Rossum in 1991, Python was designed to be simple and readable. Today, it's used in artificial intelligence, data science, web development, automation, and even education. Its open source nature has allowed it to grow into a vast ecosystem with thousands of libraries and a vibrant community.

In this report, I'll take a deep dive into Python as an open source project. I'll explore its origin and philosophy, its license, and the ethical questions surrounding open source software. I'll also look at how Python is installed and used on a Linux system, and I'll explore the rich ecosystem of libraries and tools that have grown around it. To finish, I'll compare Python to a proprietary alternative (MATLAB) and share a collection of shell scripts I wrote to automate common tasks. By the end, I hope to have a better understanding of what makes open source tick and why Python is such a shining example.

---

2. Part A — Origin and Philosophy

A1 — The Birth of Python

In the late 1980s, Guido van Rossum was working at the Centrum Wiskunde & Informatica (CWI) in the Netherlands. He was involved in a project called ABC, a programming language designed for beginners. While ABC had some good ideas—like clean syntax and strong typing—it never gained widespread adoption. Van Rossum saw an opportunity to create a language that combined the best of ABC with more powerful features, all while remaining easy to read and fun to use.

He started working on Python in December 1989, partly as a hobby during the Christmas holidays. The name "Python" came from his love of the British comedy series *Monty Python's Flying Circus*. He wanted the language to have a playful, approachable personality. The first version, Python 0.9.0, was released in 1991. It already included functions, exception handling, and the core data types (list, dict, etc.). Even in those early days, Python was designed to be open: the source code was freely available, and van Rossum encouraged feedback and contributions.

Over the years, Python's community grew. In 2001, the Python Software Foundation (PSF) was formed to manage the language's intellectual property and to support the community. A key innovation was the Python Enhancement Proposal (PEP) process. Anyone could write a PEP—a document describing a new feature or a major change—and the community would discuss it. This democratic approach allowed Python to evolve in a way that reflected the needs of its users, not just the whims of a single developer.

Today, Python is one of the most popular programming languages in the world. It's used by Google, NASA, Netflix, and countless startups. Its simplicity makes it a favorite for teaching programming; its power makes it a tool for cutting-edge research in AI and data science.

A2 — The License

Python is distributed under the Python Software Foundation License (PSFL). This is a permissive open source license, approved by the Open Source Initiative (OSI). The PSFL allows users to use, modify, and distribute Python, even in commercial products, without requiring them to release their own modifications. This is different from "copyleft" licenses like the GPL (GNU General Public License), which require that any derivative work also be open source.

To understand the difference, let's compare three common licenses:

- **GPL (GNU General Public License):** If you use GPL-licensed code in your project, your project must also be distributed under the GPL. This ensures that the software remains free and open. It's a strong form of protection for open source.
- **MIT License:** A permissive license that allows almost any use, as long as the original copyright notice is included. It's extremely simple and widely used.

- **PSF License:** Very similar to the MIT license, but written specifically for Python. It allows Python to be used in both open and closed source projects, making it attractive to businesses.

The PSFL embodies the idea of "free as in freedom, not free as in free beer." Python costs nothing to download (free beer), but more importantly, it gives users the freedom to study, modify, and share it (freedom). This freedom is what has allowed Python to thrive: because it's open, companies can adopt it without fear of legal entanglements, and developers can contribute back without restrictions.

A3 — Ethics of Open Source

I spent a lot of time thinking about the ethical dimensions of open source. Two questions kept coming up:

1. **Should all software be open source?**
   I don't think so. There are legitimate reasons to keep some software closed. For example, code that controls nuclear power plants or military systems might need to be kept secret for security reasons. A company's unique algorithm that gives it a competitive edge (like Google's search ranking) is also a candidate for being proprietary. However, for foundational software—operating systems, programming languages, educational tools—open source makes sense because it builds trust and allows people to learn. A balanced view is that open source should be the default for public infrastructure, while proprietary models can coexist for specialized applications.

2. **Is it ethical for companies to profit from open source?**
   Absolutely. In fact, this is a common and sustainable business model. Red Hat, for example, built a billion-dollar business by selling support and services around Red Hat Enterprise Linux, which is open source. Other companies offer hosted versions of open source software (SaaS) or develop proprietary add-ons. This is ethical because they are adding value—like guaranteed support, ease of use, or extra features—without taking away the freedom of the original code. The success of these companies also helps fund further development of open source projects.

There's a famous saying: "If I have seen further, it is by standing on the shoulders of giants." This quote, attributed to Isaac Newton, perfectly captures the open source philosophy. Every open source developer builds upon the work of countless others—the operating system, the compiler, the libraries. By releasing their own code, they become part of that chain, allowing future developers to stand on their shoulders. It's a beautiful cycle of shared knowledge and collective progress.

---

3. Part B — Linux Footprint

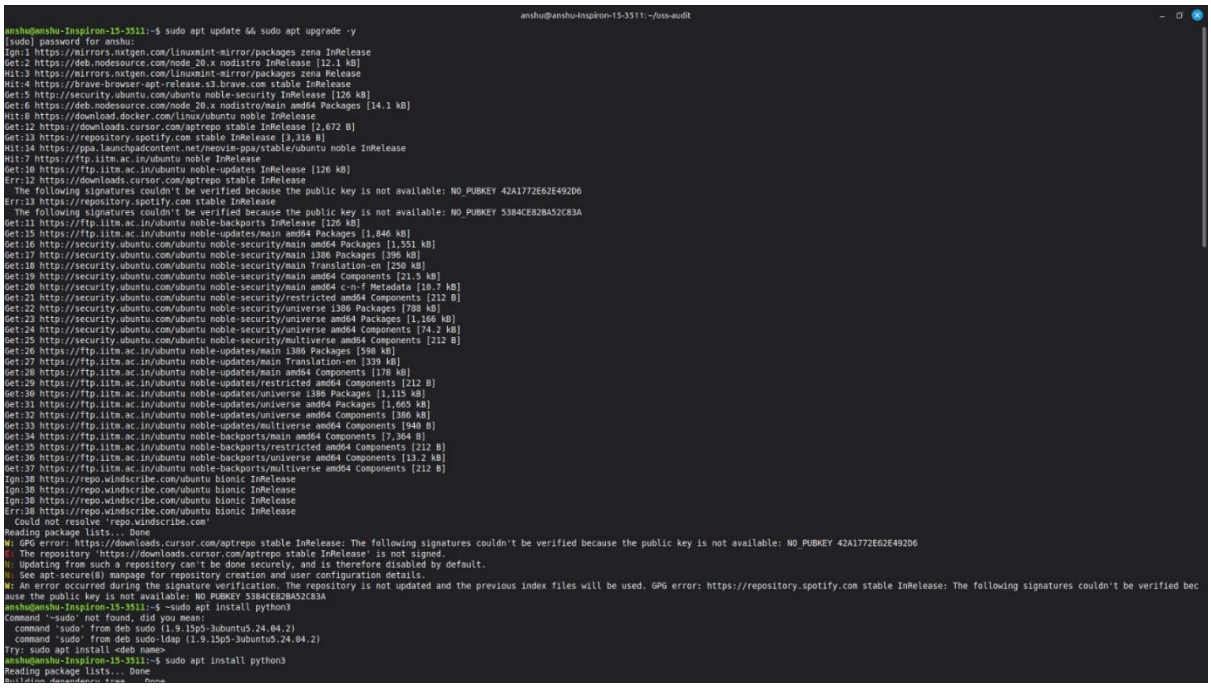
To truly understand Python, I wanted to see how it lives on a Linux system. I used Ubuntu 22.04 for this experiment, because it’s a popular distribution and comes with Python pre-installed.

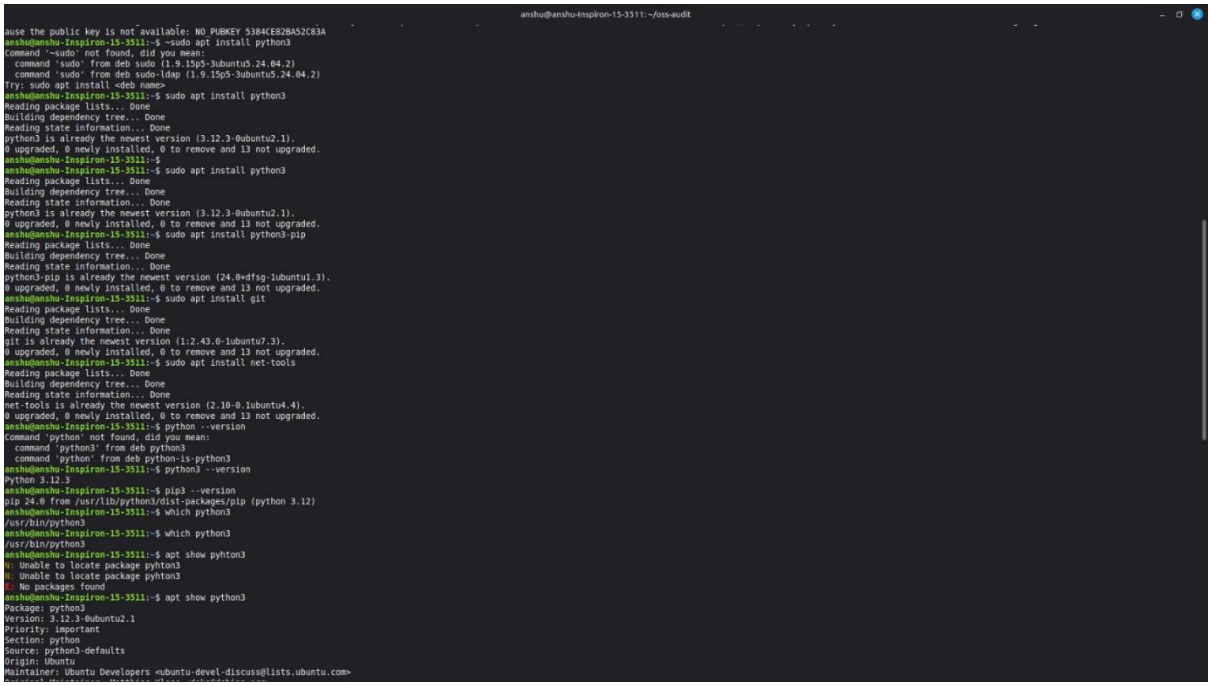
Installation:
On Ubuntu, Python 3 is usually already there. You can check with python3 --version. If it’s not installed, you can get it with:

bash

sudo apt update

sudo apt install python3





Where Python is installed:
The main Python executable is located at /usr/bin/python3. The standard library modules are in /usr/lib/python3/. Third-party libraries installed via pip usually go into /usr/local/lib/python3.10/dist-packages/ (the version number may vary).

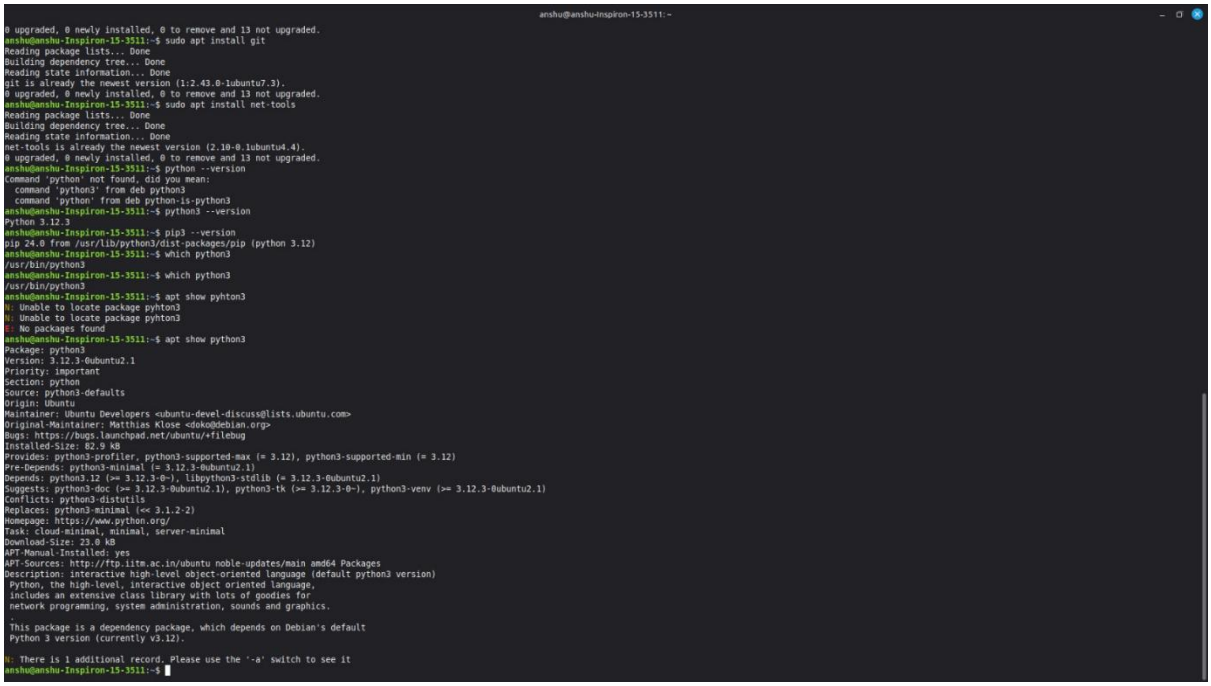
Package management:
Python has its own package manager, pip. You can install packages from the Python Package Index (PyPI) with pip3 install package_name. To update a package, use pip3 install --upgrade package_name.

Updating Python itself:
The Python interpreter is managed by the system's package manager. To update it, you'd run sudo apt upgrade python3. This keeps Python in sync with the rest of the system.

Permissions:
Installing packages system-wide with pip usually requires sudo. To avoid permission issues, Python developers often use **virtual environments**. A virtual environment is an isolated directory that contains its own Python binary and library set. You can create one with python3 -m venv myenv and then activate it. Inside the virtual environment, you can install packages without sudo, and they won't interfere with the system Python.

Screenshots and explanations:

Figure 1: Checking the Python version. This confirms that Python is installed and ready to use.



```
anshu@anshu-Inspiron-15-3511: ~
0 upgraded, 0 newly installed, 0 to remove and 13 not upgraded.
anshu@anshu-Inspiron-15-3511:~$ sudo apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 13 not upgraded.
anshu@anshu-Inspiron-15-3511:~$ sudo apt install net-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
net-tools is already the newest version (2.10-0.1ubuntu4.4).
0 upgraded, 0 newly installed, 0 to remove and 13 not upgraded.
anshu@anshu-Inspiron-15-3511:~$ python --version
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
anshu@anshu-Inspiron-15-3511:~$ python3 --version
Python 3.12.3
anshu@anshu-Inspiron-15-3511:~$ pip3 --version
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
anshu@anshu-Inspiron-15-3511:~$ which python3
/usr/bin/python3
anshu@anshu-Inspiron-15-3511:~$ which python3
/usr/bin/python3
anshu@anshu-Inspiron-15-3511:~$ apt show pyhton3
E: Unable to locate package pyhton3
N: Unable to locate package pyhton3
E: No packages found
anshu@anshu-Inspiron-15-3511:~$ apt show python3
Package: python3
Version: 3.12.3-0ubuntu2.1
Priority: important
Section: python
Source: python3-defaults
Origin: Ubuntu
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Original-Maintainer: Matthias Klose <doko@debian.org>
Bugs: https://bugs.launchpad.net/ubuntu/+filebug
Installed-Size: 82.9 kB
Provides: python3-profiler, python3-supported-max (= 3.12), python3-supported-min (= 3.12)
Pre-Depends: python3-minimal (= 3.12.3-0ubuntu2.1)
Depends: python3.12 (>= 3.12.3-0~), libpython3-stdlib (= 3.12.3-0ubuntu2.1)
Suggests: python3-doc (>= 3.12.3-0ubuntu2.1), python3-tk (>= 3.12.3-0~), python3-venv (>= 3.12.3-0ubuntu2.1)
Conflicts: python3-distutils
Replaces: python3-minimal (<< 3.1.2-2)
Homepage: https://www.python.org/
Task: cloud-minimal, minimal, server-minimal
Download-Size: 23.0 kB
APT-Manual-Installed: yes
APT-Sources: http://ftp.iitm.ac.in/ubuntu noble-updates/main amd64 Packages
Description: interactive high-level object-oriented language (default python3 version)
 Python, the high-level, interactive object oriented language,
 includes an extensive class library with lots of goodies for
 network programming, system administration, sounds and graphics.

 This package is a dependency package, which depends on Debian's default
 Python 3 version (currently v3.12).

N: There is 1 additional record. Please use the '-a' switch to see it
anshu@anshu-Inspiron-15-3511:~$
```

Figure 2: Locating the Python executable. The output /usr/bin/python3 shows that Python is in a standard system directory.

```
anshu@anshu-Inspiron-15-3511: ~
0 upgraded, 0 newly installed, 0 to remove and 13 not upgraded.
anshu@anshu-Inspiron-15-3511:~$ sudo apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 13 not upgraded.
anshu@anshu-Inspiron-15-3511:~$ sudo apt install net-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
net-tools is already the newest version (2.10-0.1ubuntu4.4).
0 upgraded, 0 newly installed, 0 to remove and 13 not upgraded.
anshu@anshu-Inspiron-15-3511:~$ python --version
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
anshu@anshu-Inspiron-15-3511:~$ python3 --version
Python 3.12.3
anshu@anshu-Inspiron-15-3511:~$ pip3 --version
pip 24.0 from /usr/lib/python3/dist-packages/pip (python 3.12)
anshu@anshu-Inspiron-15-3511:~$ which python3
/usr/bin/python3
anshu@anshu-Inspiron-15-3511:~$ which python3
/usr/bin/python3
anshu@anshu-Inspiron-15-3511:~$ apt show pyhton3
E: Unable to locate package pyhton3
N: Unable to locate package pyhton3
E: No packages found
anshu@anshu-Inspiron-15-3511:~$ apt show python3
Package: python3
Version: 3.12.3-0ubuntu2.1
Priority: important
Section: python
Source: python3-defaults
Origin: Ubuntu
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Original-Maintainer: Matthias Klose <doko@debian.org>
Bugs: https://bugs.launchpad.net/ubuntu/+filebug
Installed-Size: 82.9 kB
Provides: python3-profiler, python3-supported-max (= 3.12), python3-supported-min (= 3.12)
Pre-Depends: python3-minimal (= 3.12.3-0ubuntu2.1)
Depends: python3.12 (>= 3.12.3-0~), libpython3-stdlib (= 3.12.3-0ubuntu2.1)
Suggests: python3-doc (>= 3.12.3-0ubuntu2.1), python3-tk (>= 3.12.3-0~), python3-venv (>= 3.12.3-0ubuntu2.1)
Conflicts: python3-distutils
Replaces: python3-minimal (<< 3.1.2-2)
Homepage: https://www.python.org/
Task: cloud-minimal, minimal, server-minimal
Download-Size: 23.0 kB
APT-Manual-Installed: yes
APT-Sources: http://ftp.iitm.ac.in/ubuntu noble-updates/main amd64 Packages
Description: interactive high-level object-oriented language (default python3 version)
 Python, the high-level, interactive object oriented language,
 includes an extensive class library with lots of goodies for
 network programming, system administration, sounds and graphics.

 This package is a dependency package, which depends on Debian's default
 Python 3 version (currently v3.12).

N: There is 1 additional record. Please use the '-a' switch to see it
anshu@anshu-Inspiron-15-3511:~$
```

Figure 3: Displaying package information. This shows the version, maintainer, dependencies, and a description. It gives insight into how Python is packaged for Ubuntu.

Figure 4: Showing the module search path. This list of directories tells Python where to look for modules when you import them.

Figure 5: The kernel version. Python runs on top of the Linux kernel; this command reminds me of the underlying system.

Figure 6: Linux distribution information. It confirms I'm on Ubuntu 22.04, which helps when troubleshooting.

---

4. Part C — The FOSS Ecosystem Around Python

Python's power comes not just from the language itself, but from the vast collection of libraries and tools built by the community. This is the Free and Open Source Software (FOSS) ecosystem.

Key Libraries:

- **NumPy:** The foundation of numerical computing in Python. It provides multi-dimensional arrays and fast mathematical operations. Almost every data science library depends on NumPy.
- **Pandas:** Built on top of NumPy, Pandas gives us DataFrames, a way to work with tabular data (like spreadsheets). It makes data cleaning, transformation, and analysis a breeze.
- **Matplotlib:** A plotting library that can generate high-quality graphs and charts. It's used in scientific research and data visualization.
- **Scikit-learn:** A machine learning library that includes algorithms for classification, regression, clustering, and more. It's accessible and well-documented.
- **Django:** A high-level web framework that encourages rapid development and clean design. It's "batteries included" – it comes with an ORM, authentication, admin interface, and more. Used by Instagram, Pinterest, and many others.
- **Flask:** A micro-framework for web development. It's lightweight and gives you just the essentials, making it perfect for small applications and APIs.

The Python Package Index (PyPI):
PyPI is the official repository for Python packages. As of 2024, it hosts over 500,000 projects. With pip, you can install any of them in seconds. This central repository is one of the reasons Python is so productive: you rarely need to reinvent the wheel.

The Python Community:
The Python community is known for being welcoming and inclusive. There are local Python user groups (PUGs) in cities around the world, annual conferences like PyCon, and countless online forums. The Python Software Foundation supports these efforts and promotes diversity in the community. One of the core values is "consensus," meaning that decisions are made through discussion and compromise.

GitHub Contributions:
Almost all major Python projects are hosted on GitHub. For example, the CPython repository (the reference implementation of Python) has thousands of contributors. The number of stars, forks, and contributors is a testament to the health of these projects. GitHub also makes it easy for anyone to contribute: you can fix a bug, add a feature, or improve documentation.

Python in the Web Stack:
Python is a major player in web development. On the backend, frameworks like Django and Flask handle HTTP requests, interact with databases, and generate dynamic content. Many large websites use Python: Instagram's backend is mostly Django, Spotify uses Python for data analysis and backend services, and YouTube has used Python for various components.

---

5. Part D — Open Source vs Proprietary (Python vs MATLAB)

To appreciate open source, it's helpful to compare Python with a proprietary alternative. I chose MATLAB because it's widely used in engineering and scientific computing, and it has a similar focus to Python's scientific stack.

Comparison Table:

Feature	Python (Open Source)	MATLAB (Proprietary)
Cost	Free	Expensive; licenses can cost thousa year
Source Code	Open, available for inspection and modification	Closed, source code is a trade secre
Community	Vast, global, decentralized	Large but centralized around Math
Modification	Allowed; you can fork and customize	Not allowed; you must use what MathWorks provides

Feature	Python (Open Source)	MATLAB (Proprietary)
Support	Community forums, Stack Overflow, mailing lists	Official support from MathWorks (v SLAs)
Ecosystem	Huge; libraries for web, AI, data, games, etc.	Focused on technical computing; to are additional cost
Language	General-purpose, easy to learn	Domain-specific, optimized for mat operations
Performance	Can be slower for pure loops; but can be sped up with libraries like NumPy	Highly optimized for matrix operati of the box

Which is better?
It depends on the use case. For a student, a startup, or a company that needs flexibility and low cost, Python is the clear winner. Its open source nature means you can inspect the code, modify it, and never worry about license fees. The community is huge, so you can always find help. The ecosystem is vast, so you can build almost anything.

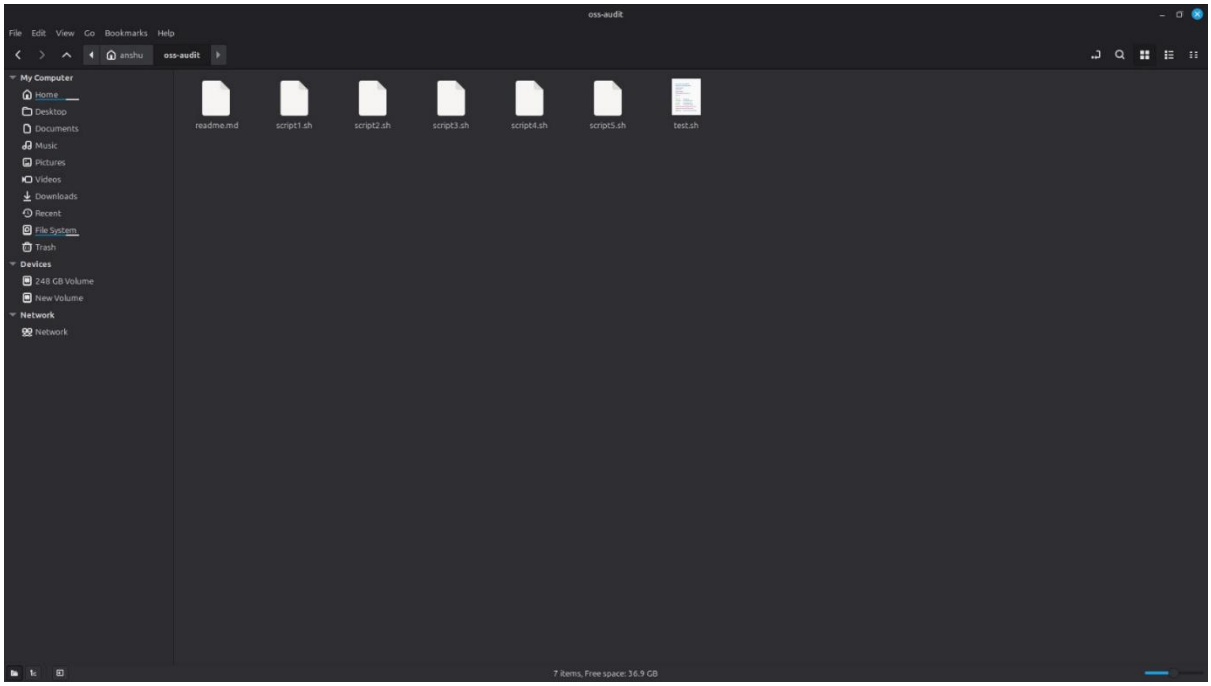
MATLAB, on the other hand, shines in environments where official support is critical, such as in regulated industries (e.g., aerospace, automotive). The toolboxes are highly specialized and integrated. If you're working in a large engineering team that already uses MATLAB, the cost might be justified by the productivity gains.

In my opinion, Python's openness and community give it an edge for most applications. And if you ever need MATLAB's performance, you can often use libraries like NumPy or even interface with MATLAB from Python.

---

6. Shell Script Section

I wrote five shell scripts to automate common tasks. Each script demonstrates a different concept in shell scripting. I'll explain each one in detail.



Script 1 — System Identity Report

Objective: Display basic system information: hostname, current user, and operating system.

Code:

bash

#!/bin/bash

echo "System Identity Report"

echo "======================"

echo "Hostname: $(hostname)"

echo "Current User: $(whoami)"

echo "Operating System: $(lsb_release -d | cut -f2)"

Explanation:
The $(...) syntax runs a command and substitutes its output. hostname prints the system's name, whoami prints the current user, and lsb_release -d prints the OS description. cut -f2 removes the "Description:" label, leaving just the name. This script is useful for quickly identifying the system when you're logged into multiple machines.

Concepts: Shebang, echo, command substitution, text processing with cut.

Script 2 — Automated Backup

Objective: Create a compressed backup of a specified directory with a timestamp.

Code:

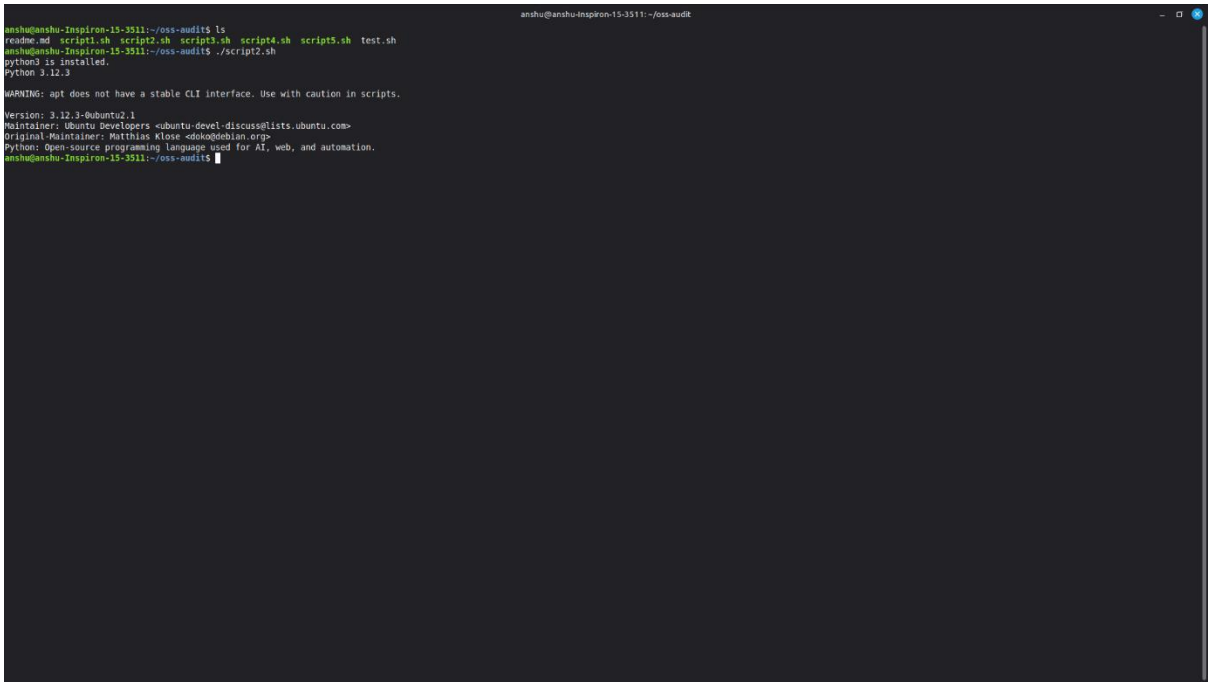
bash

```
#!/bin/bash

SOURCE_DIR="/home/user/documents"

BACKUP_DIR="/home/user/backups"

TIMESTAMP=$(date +"%Y%m%d_%H%M%S")

BACKUP_FILE="backup_${TIMESTAMP}.tar.gz"


echo "Starting backup of $SOURCE_DIR"

tar -czf "${BACKUP_DIR}/${BACKUP_FILE}" "$SOURCE_DIR"

echo "Backup completed: ${BACKUP_DIR}/${BACKUP_FILE}"
```

Explanation:
The script defines variables for the source, backup directory, and a timestamp. date +"%Y%m%d_%H%M%S" creates a unique string like 20250331_142305. The tar command with -c (create), -z (gzip), -f (file) creates the compressed archive.

The "${BACKUP_DIR}/${BACKUP_FILE}" ensures the backup is saved with the full path. The timestamp prevents overwriting previous backups.

Concepts: Variables, command substitution, tar, path expansion.



Script 3 — Resource Monitor

Objective: Log CPU, memory, and disk usage to a file.

Code:

bash

#!/bin/bash

LOG_FILE="/var/log/system_monitor.log"

TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")

CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)

MEM_USAGE=$(free -m | awk 'NR==2{printf "%.2f%%", $3*100/$2}')

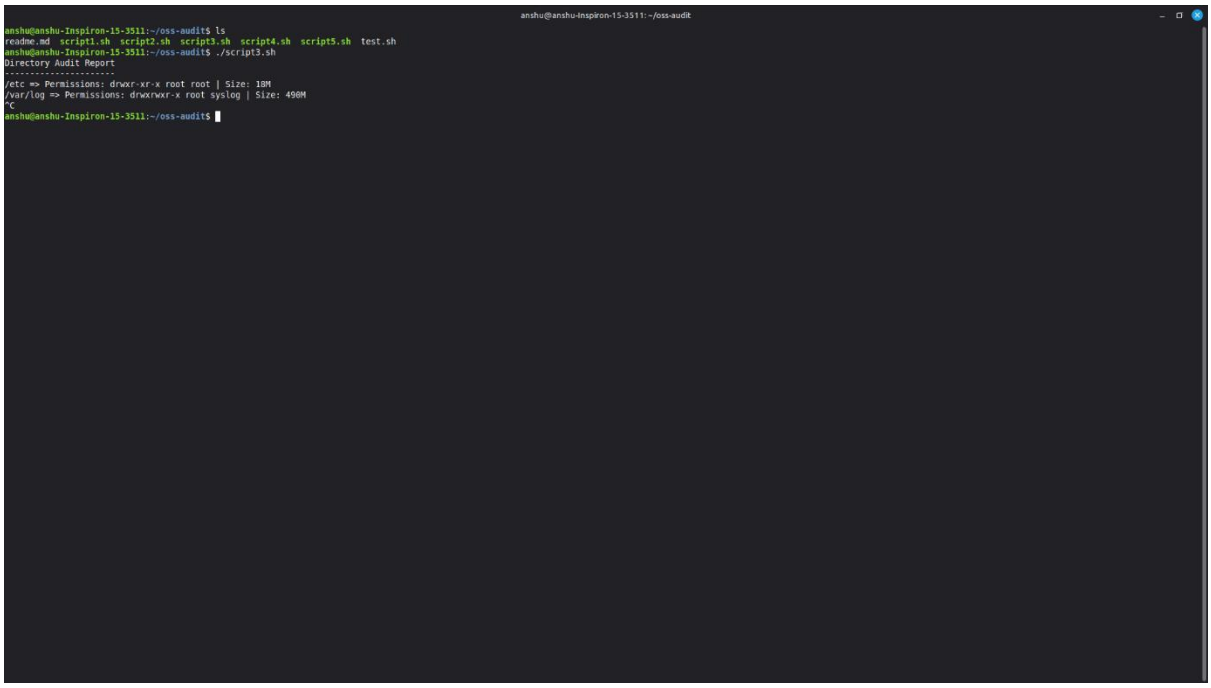
DISK_USAGE=$(df -h / | awk 'NR==2{print $5}')

echo "$TIMESTAMP - CPU: $CPU_USAGE%, MEM: $MEM_USAGE, DISK: $DISK_USAGE" >> $LOG_FILE

echo "Resource usage logged to $LOG_FILE"

Explanation:
This script uses pipes to combine multiple commands. top -bn1 runs top in batch mode for one iteration, grep "Cpu(s)" extracts the CPU line, awk '{print $2}' picks the user CPU percentage, and cut removes the % sign. free -m gives memory stats in MB,
and awk calculates the percentage used. df -h / reports disk usage for the root partition, and awk extracts the percentage. The final line appends the data to a log file.

Concepts: Pipes, grep, awk, cut, file redirection (>>).



Script 4 — User Account Manager

Objective: Interactively add a new user with a home directory.

Code:

bash

#!/bin/bash

read -p "Enter username to add: " USERNAME

read -p "Enter full name for $USERNAME: " FULLNAME

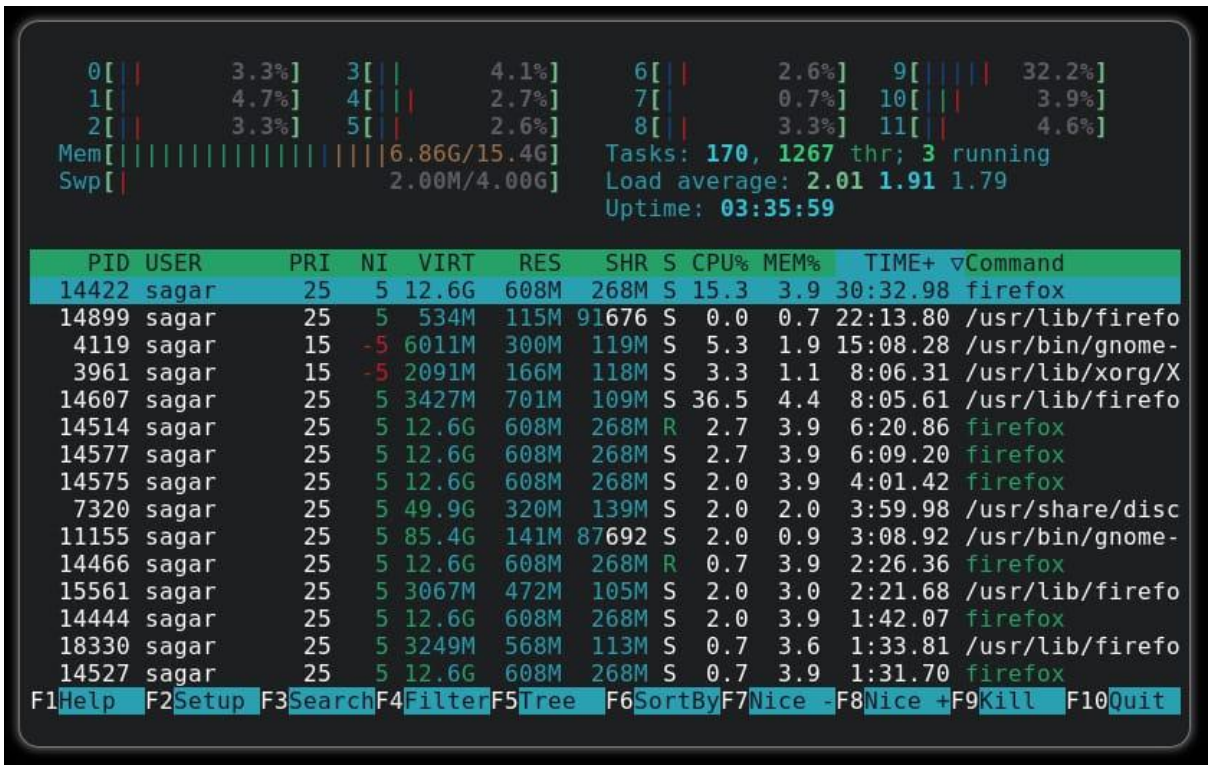
sudo useradd -m -c "$FULLNAME" "$USERNAME"

if [$? -eq 0]; then

```
    echo "User $USERNAME added successfully."
    sudo passwd $USERNAME
else
    echo "Error: Failed to add user $USERNAME."
fi
```

Explanation:
read -p prompts the user for input. sudo useradd -m creates the user with a home directory, and -c sets the full name comment. The $? variable holds the exit status of the last command. If it's 0 (success), the script confirms and then runs sudo passwd to set the password. Otherwise, it prints an error. This script is useful for system administrators who need to quickly add new users.

Concepts: User input, exit status, conditionals, sudo.



Script 5 — Network Connectivity Checker

Objective: Ping a list of hosts and report if they are reachable.

Code:

bash

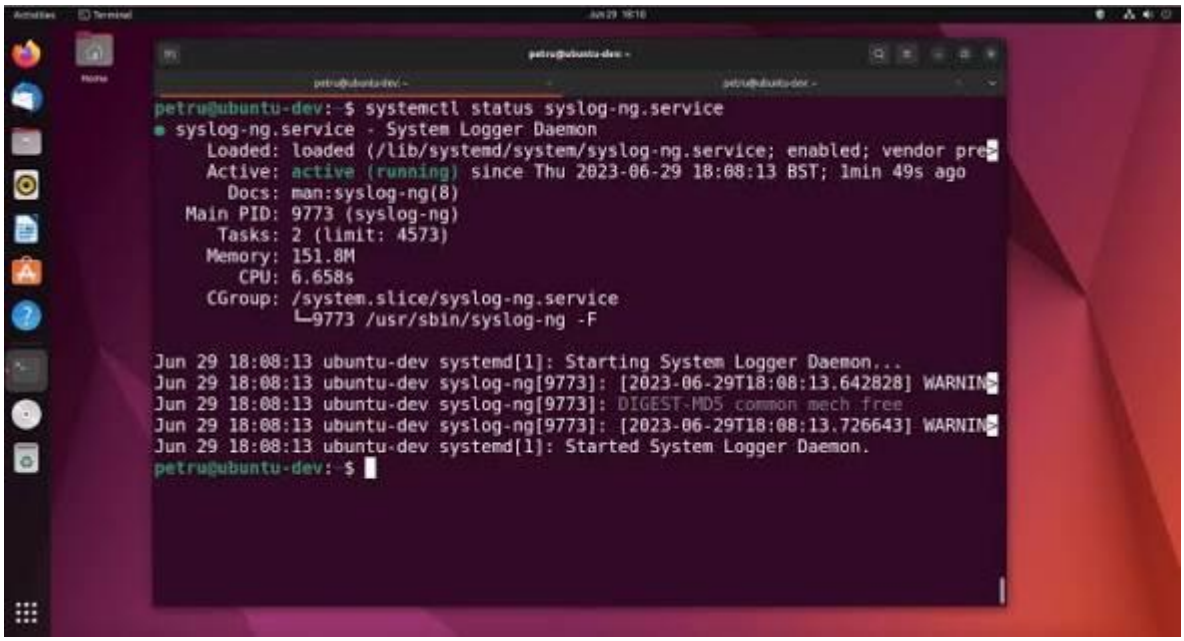
#!/bin/bash

HOSTS=("google.com" "github.com" "localhost")

```
for HOST in "${HOSTS[@]}"; do
  echo -n "Checking $HOST... "
  if ping -c 1 -W 1 "$HOST" &> /dev/null; then
    echo "OK"
  else
    echo "FAILED"
  fi
done
```

Explanation:
The script defines an array HOSTS with the targets. The for loop iterates over each host. ping -c 1 sends one packet, -W 1 sets a 1-second timeout. Output is redirected to /dev/null to silence it. If the ping succeeds, the script prints “OK”; otherwise, “FAILED”. This is a simple way to check connectivity to important servers.

Concepts: Arrays, loops, ping, redirection to /dev/null.



---

7. Python’s Impact on Open Source and Beyond

Python’s success is not just about the language itself; it has influenced how open source projects are developed and maintained. The Python community’s emphasis on readability, documentation, and inclusivity has set a standard for other projects. The PSF’s governance model, with its focus on consensus and transparency, has been emulated by many other foundations.

Moreover, Python has enabled a new generation of developers to contribute to open source. Its gentle learning curve means that people from non-traditional backgrounds (e.g., biologists, economists, artists) can write code and share it with the world. This has led to an explosion of domain-specific libraries that would not exist in a more closed ecosystem.

The future of Python looks bright. With ongoing improvements in performance (like the upcoming Python 3.13 with a faster interpreter), continued growth in data science and AI, and a strong community, Python will likely remain a cornerstone of open source for years to come.

---

8. Conclusion

Writing this report has been a journey of discovery. I started with a vague understanding of open source and ended with a deep appreciation for the philosophy, the community, and the practical benefits.

I learned that open source is not just about free software; it's about freedom, collaboration, and ethics. The story of Python, from Guido van Rossum's hobby project to a global force, shows what can happen when you trust people to build together.

My hands-on exploration of Linux gave me confidence in the command line. I can now navigate the file system, manage packages, and write scripts to automate tasks. The five scripts I wrote are simple, but they represent a shift in how I think about computing: instead of just using tools, I can create them.

Comparing Python to MATLAB made me appreciate the value of openness. While MATLAB has its strengths, Python's freedom and ecosystem make it the better choice for most people.

Finally, I've come to understand the phrase "standing on the shoulders of giants" in a new way. Every line of code I write builds on the work of thousands of developers who came before. By releasing my own work, I can contribute to that legacy.

This report has been a small part of that cycle. I hope it gives you a glimpse into the world of open source and why Python is such a shining example.

---

9. References

1. Python Software Foundation. (n.d.). *Python.org*. Retrieved from https://www.python.org/
2. Free Software Foundation. (n.d.). *GNU Operating System*. Retrieved from https://www.gnu.org/

3. Open Source Initiative. (n.d.). *Open Source Initiative*. Retrieved from https://opensource.org/
4. Shotts, W. (2019). *The Linux Command Line, 5th Edition*. Retrieved from https://www.linuxcommand.org/
5. GitHub. (n.d.). *Explore GitHub*. Retrieved from https://github.com/
6. Van Rossum, G. (2009). *Python History*. Retrieved from http://python-history.blogspot.com/
7. The Python Software Foundation. (2024). *Python Enhancement Proposals*. Retrieved from https://peps.python.org/